

Teaching Statement

Tien Soare (Nguyen)

Teaching Philosophy

I believe the central skill in computer science is reasoning about what a program actually does, as opposed to what we hope it does. Students who learn to reason this way can debug, design, and verify code on their own; students who never do are left guessing. My goal as a teacher is to make program behavior visible, so that every student, and especially those writing code for the first time, can build an accurate mental model of execution and trust it. This goal shows up in my teaching as three concrete commitments.

Make execution visible. Most students who struggle are not missing ability; they are missing a view of what their program is doing. I break problems into small steps and make each step's effect on program state explicit: drawing the call stack as a recursive function unwinds, or filling in a variable table one loop iteration at a time.

Let students do the reasoning. A fixed bug teaches far less than a found one. Rather than giving answers, I ask students to predict what their code will do and then check that prediction against what actually happens. The gap between the two is where learning happens.

Treat confusion as normal. The students who most need help are often the least likely to ask for it. I began my own computer science education at a community college, and I know how much it matters when a teacher makes it safe to say “I don't understand.” I listen for the misconception underneath a question and respond to that, not just to the surface error.

These commitments matter even more now that students have access to large language models. My research studies where LLMs succeed and fail at reasoning about code, and it has convinced me that the skills an LLM cannot substitute for—tracing execution, predicting output, and judging whether code is correct—are exactly the skills an introductory course must protect and teach.

Teaching Experience

Since Fall 2023, I have served as a Graduate Teaching Assistant at Virginia Tech for five semesters across two introductory courses: CS 1064 (Introduction to Programming in Python; Fall 2024, Spring 2025, and Summer 2026) and CS 1114 (Introduction to Software Design; Fall 2023 and Fall 2025). In CS 1064, I led lab sections and guided students through their first programming experience; in CS 1114, I supported students learning object-oriented programming through labs and one-on-one debugging sessions. In both courses, I held regular office hours and graded assignments. Below are the practices I have developed in those roles and what I have learned from them.

Predict-then-run office hours

When a student brings broken code, I do not point to the bug. I ask them to walk me through the program line by line and say what each line should do before we run it. The bug almost always appears at the first line where their prediction and the program disagree, and they find it themselves. Over a semester, I have watched students shift from making random edits until the tests pass to tracing their code *before* coming to office hours. Many students arrive expecting me to hand them the answer. Instead, I ask what they tried, what they expected to happen, and which concept they were applying. These questions usually reveal the specific idea they misunderstood, and once it is named, I guide them to see how it applies to their problem, so that the fix comes from them rather than from me.

Tracing and visualization for hard topics

Loops, recursion, and object references are where introductory students most often lose their footing. For these topics, I start from the smallest case and grow it, using hand-drawn sketches of memory alongside the built-in debuggers of IDEs such as Eclipse, IntelliJ, and BlueJ, which let students step through execution and watch variables and object state change line by line. For object-oriented concepts in CS 1114, I pair a concrete metaphor with the corresponding code so that students do not leave with the metaphor alone.

Teaching that correct-looking code can be wrong

My research on floating-point numerical stability gives me a favorite classroom moment: in Python, `0.1 + 0.2 == 0.3` evaluates to `False`. Students are always surprised, and that surprise opens a discussion about the difference between “the code runs” and “the code is correct.” I use such examples to build the habit of questioning output rather than accepting it.

Student feedback

I have not received formal teaching evaluations as a TA, but students have consistently given me direct verbal feedback. Students have told me that my explanations and visual sketches are easy to follow and helped them understand material they had not fully grasped in lecture. Many students returned to my office hours in particular, telling me they felt comfortable talking through their confusion with me. I take this feedback seriously, and it has shaped how I pace my explanations and how much I let students drive our sessions.

Pedagogical training

I am completing Virginia Tech’s Future Professoriate Graduate Certificate, through which I have studied inclusive pedagogy, course design and assessment, and teaching methodologies. As a faculty member, I will add structured mid-semester feedback to my courses so that I can measure, not only hear, what is working.

Courses I Can Teach

Introductory programming (CS1/CS2, Python or Java). This is where my teaching experience is deepest. I would build these courses around execution tracing, frequent low-stakes assessments that surface misconceptions early, and assignments that ask students to explain why their code works, not only to submit passing output. Given the availability of LLMs, I would include code-reading and output-prediction questions that students must answer without assistance, and designated assignments where students use an LLM and then critique what it produced.

Software engineering. Drawing on my TA experience in CS 1114 and my research in software reliability, I would emphasize testing, debugging, and code review as engineering disciplines rather than afterthoughts. Team projects would include explicit verification milestones, and students would learn to evaluate AI-generated code with the same rigor they apply to their teammates’ code.

LLM-based program analysis in the AI era. This course, suitable as an advanced undergraduate elective or a graduate seminar, grows directly out of my research. It would cover classical program analysis alongside the use of LLMs for bug detection, program repair, and numerical reasoning, with a strong focus on where these models fail and how to evaluate them rigorously. Students would read current research, reproduce published results, and complete a project that applies or critiques an LLM-based analysis technique. My goal is to train students who can use these tools effectively precisely because they understand their limits.

Across all of these courses, my aim is the same: a classroom that is rigorous and supportive at once, where students leave able to reason through problems they have never seen before.